Smart Campus Canteen Food Demand Prediction using Machine Learning (Random Forest Regression)

Problem Statement:

University and college canteens must decide every morning how much food to prepare for several thousand students, yet the actual demand changes from day to day. It rises on busy working days, exam weeks and campus-event days, falls sharply on Sundays, holidays and vacations, and shifts with the weather and with promotions. When the kitchen prepares more than is sold, cooked food is wasted and money is lost; when it prepares less, students face queues, empty counters and lower satisfaction. Estimates made by canteen staff from experience are subjective and do not systematically use the information that is already available, such as the academic calendar, the weather and recent sales.

There is, therefore, a need for a data-driven system that can estimate, in advance, the number of units of every menu item that will be sold on a given day, so that the quantity to be cooked can be planned with less waste and fewer shortages. This project designs and implements a Smart Campus Canteen Food Demand Prediction system in Python 3 using JupyterLab. It performs data cleaning, calendar, weather and lag-based feature engineering, and regression using a Random Forest model tuned with time-series cross-validation, and evaluates the trained model on an unseen, later period using standard regression metrics (MAE, RMSE, R^2 Score, WAPE), a comparison with baseline models, and an estimate of surplus and shortage units.

Objectives:

1. To collect and organise a daily canteen sales dataset containing the menu item, price, promotion, weather, students on campus, calendar flags and units sold.
2. To clean the data by removing duplicates, repairing missing weather readings and validating the target variable.
3. To engineer meaningful features (calendar features, previous-day and previous-week demand, 7-day rolling mean, one-hot encoded menu items) from the cleaned data.
4. To design, tune (time-series cross-validated grid search), train and save a Random Forest Regression model for daily demand.
5. To evaluate the model on an unseen, chronologically later test period and report MAE, RMSE, R^2 and WAPE, compare it with baseline models, and estimate surplus and shortage units.
6. To persist every prediction (forecast date, item, predicted units, actual units, error, timestamp) in an SQLite database for record-keeping.

Software / Hardware Required:

• Operating System: Windows / Linux / macOS (executed on Ubuntu Linux)

• Environment: JupyterLab (notebook: notebooks/Canteen_Demand_Prediction.ipynb)

• Language: Python 3.x (executed with Python 3.12.3)

• Machine Learning: scikit-learn 1.8.0 (RandomForestRegressor, GridSearchCV, TimeSeriesSplit)

• Data Handling: pandas 3.0.2, NumPy 2.4.4

• Visualization: Matplotlib 3.10.8

• Model Persistence: joblib 1.5.3

• Database: SQLite3 (prediction log)

• Dataset: Synthetic daily canteen sales data – 3,655 records (731 days × 5 menu items, 2023-07-01 to 2025-06-30) generated by data/generate_dataset.py with a fixed random seed (42)

• Hardware: Standard CPU-only computer (executed on 1 CPU core, 3 GB RAM); no GPU required

Data note: no real canteen records were available, so the dataset is synthetic and is produced from explicit, documented rules (attendance, weekday, weather, exams, events, promotions and random noise). All results in this report therefore describe performance on simulated data. Replacing data/canteen_demand.csv with real records that have the same columns allows the same code to be retrained without modification.

Project Directory Structure:

The project ("CanteenDemandPrediction") is organized with separate folders for the dataset, source modules, trained model, notebook, outputs, database and screenshots, as shown below.

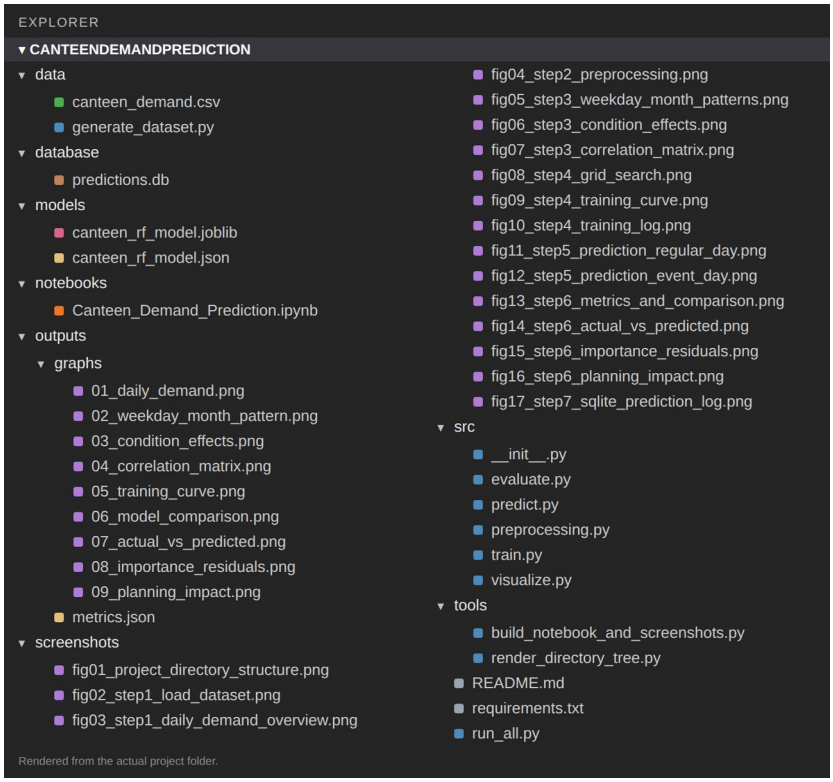


Fig 1: Project directory structure of CanteenDemandPrediction (generated from the actual project folder)

Procedure:

1. Prepare the dataset: run data/generate_dataset.py to create data/canteen_demand.csv (daily sales per menu item with price, promotion, weather, attendance and calendar flags).

2. Install the libraries listed in requirements.txt and launch JupyterLab; open notebooks/Canteen_Demand_Prediction.ipynb.

3. Load the CSV file and inspect its shape, data types, summary statistics and the daily demand trend (Step 1).

4. Clean the data – remove duplicate records, repair the temperature (linear interpolation) and rainfall (monthly median) sensor gaps and clip negative sales to zero (Step 2).

5. Engineer features – calendar features, previous-day and same-weekday-last-week demand, a shifted 7-day rolling mean and one-hot encoded menu items (20 features) – and analyse weekday, seasonal, weather, event and correlation patterns (Step 3).

6. Split the data chronologically (training: 2023-07-08 to 2025-02-05; testing: 2025-02-06 to 2025-06-30) and tune the Random Forest using GridSearchCV with 5-fold TimeSeriesSplit (Step 4).

7. Train the final RandomForestRegressor with the best parameters (200 trees, maximum depth 10), plot the training/validation error against the number of trees and save the model to models/canteen_rf_model.joblib (Step 4).

8. Feed the feature rows of a chosen day to the trained model to obtain the predicted units for each menu item and compare them with the actual sales (Step 5).

9. Evaluate the model on the held-out test period – MAE, RMSE, R^2, WAPE, feature importance, residuals, baseline comparison and surplus/shortage analysis (Step 6).

10. Log every prediction (forecast date, item, predicted units, actual units, absolute error, timestamp) into the SQLite database database/predictions.db and display the prediction history (Step 7).

Implementation Code:

Core modules of the project (identical to the source files in the project ZIP). The plotting module (src/visualize.py), the command-line runner (run_all.py) and the notebook are provided in the ZIP.

1. Synthetic Dataset Generation (data/generate_dataset.py)

```
"""Generate a reproducible SYNTHETIC dataset of daily campus-canteen sales.

No real canteen records are used. Every row is produced from the explicit
rules below with a seeded NumPy generator, so the CSV is fully reproducible.
Replace data/canteen_demand.csv with real records (same columns) to retrain.

Run:  python data/generate_dataset.py
"""
from pathlib import Path
import numpy as np
import pandas as pd

SEED = 42
START, END = "2023-07-01", "2025-06-30"
BASE_STUDENTS = 4500

# item: (meal period, base daily demand, base price INR, rain effect per 10 mm,
#        heat effect per degC above 33)
MENU = {
    "Veg Meals":       ("Lunch",     210,  60, -0.020, -0.005),
    "Chicken Biryani": ("Lunch",     150, 120, -0.030, -0.015),
    "Masala Dosa":     ("Breakfast", 120,  45,  0.000, -0.010),
    "Idli Sambar":     ("Breakfast", 140,  30,  0.010, -0.010),
    "Samosa & Tea":    ("Snacks",    180,  25,  0.060, -0.030),
}
# Day-of-week multipliers Monday..Sunday for each meal period
DOW = {
    "Lunch":     [1.05, 1.02, 1.00, 1.00, 0.95, 0.85, 1.00],
    "Breakfast": [1.10, 1.02, 1.00, 1.00, 0.95, 0.80, 1.00],
    "Snacks":    [0.95, 1.00, 1.00, 1.02, 1.08, 1.05, 1.00],
}
FIXED_HOLIDAYS = ["01-01", "01-26", "05-01", "08-15", "10-02", "12-25"]
FESTIVAL_HOLIDAYS = ["2023-11-12", "2023-11-13", "2024-01-15", "2024-01-16",
                     "2024-10-31", "2024-11-01", "2025-01-14", "2025-01-15"]


def generate(start=START, end=END, seed=SEED):
    rng = np.random.default_rng(seed)
    dates = pd.date_range(start, end, freq="D")
    n = len(dates)
    dow, month, day, doy = dates.dayofweek.to_numpy(), dates.month.to_numpy(), \
        dates.day.to_numpy(), dates.dayofyear.to_numpy()

    # ---- calendar flags ------------------------------------------------
    public = np.array([d.strftime("%m-%d") in FIXED_HOLIDAYS or
                       d.strftime("%Y-%m-%d") in FESTIVAL_HOLIDAYS for d in dates])
    vacation = ((month == 5) & (day >= 10)) | (month == 6) | \
               ((month == 12) & (day >= 24)) | ((month == 1) & (day <= 3))
    exam = ((month == 11) & (day >= 20)) | ((month == 12) & (day <= 5)) | \
           ((month == 4) & (day >= 20)) | ((month == 5) & (day <= 9))
    event = np.zeros(n, dtype=bool)
    eligible = np.where(~vacation & ~public & (dow < 6) & ~exam)[0]
    event[rng.choice(eligible, size=20, replace=False)] = True   # ~10 per year

    # ---- weather -------------------------------------------------------
    temp = 31 + 4.5 * np.sin(2 * np.pi * (doy - 45) / 365) + rng.normal(0, 1.3, n)
    p_rain = np.where(np.isin(month, [6, 7, 8, 9]), 0.40,
                      np.where(np.isin(month, [10, 11, 12]), 0.35, 0.08))
    rain = np.where(rng.random(n) < p_rain, rng.gamma(1.6, 9.0, n), 0.0).clip(0, 120)

    # ---- students on campus --------------------------------------------
    factor = np.ones(n)
    factor[dow == 5] = 0.75
    factor[dow == 6] = 0.45
    factor[public] = np.minimum(factor[public], 0.40)
    factor[vacation] = 0.15
    factor[exam] *= 0.95
    factor[event] *= 1.20
    students = np.round(BASE_STUDENTS * factor * rng.normal(1, 0.03, n)).astype(int)

    # Sensor outages: whole-day gaps in the weather columns (handled in preprocessing)
    temp_f = temp.round(1)
    rain_f = rain.round(1)
```

```
    temp_f[rng.random(n) < 0.025] = np.nan
    rain_f[rng.random(n) < 0.020] = np.nan

    # ---- item level demand ------------------------------------------
    rows = []
    for item, (period, base, price, rain_eff, heat_eff) in MENU.items():
        promo = rng.random(n) < 0.06
        mu = base * (students / BASE_STUDENTS)
        mu = mu * np.array(DOW[period])[dow]
        mu = mu * (1 + rain_eff * np.minimum(rain, 40) / 10)
        mu = mu * (1 + heat_eff * np.maximum(temp - 33, 0))
        if period == "Breakfast":
            mu = np.where(exam, mu * 1.08, mu)
        if period == "Snacks":
            mu = np.where(exam, mu * 1.18, mu)
            mu = np.where(event, mu * 1.25, mu)
        mu = np.where(event, mu * 1.10, mu)
        mu = np.where(promo, mu * 1.15, mu)
        units = rng.poisson(mu * rng.lognormal(0, 0.06, n))
        rows.append(pd.DataFrame({
            "date": dates, "item": item, "meal_period": period,
            "price_inr": np.where(promo, price * 0.85, price).round(2),
            "is_promo": promo.astype(int),
            "temperature_c": temp_f, "rainfall_mm": rain_f,
            "students_on_campus": students,
            "is_holiday": (public | vacation).astype(int),
            "is_exam_period": exam.astype(int),
            "is_event_day": event.astype(int),
            "units_sold": units}))
    df = pd.concat(rows).sort_values(["date", "item"]).reset_index(drop=True)
    return df


if __name__ == "__main__":
    out = Path(__file__).resolve().parent / "canteen_demand.csv"
    data = generate()
    data.to_csv(out, index=False)
    print(f"Saved {len(data)} rows to {out}")
```

2. Data Cleaning and Feature Engineering (src/preprocessing.py)

```
"""Data cleaning, feature engineering and chronological train/test split."""
from dataclasses import dataclass
import numpy as np
import pandas as pd

TARGET = "units_sold"
ITEMS = ["Veg Meals", "Chicken Biryani", "Masala Dosa", "Idli Sambar", "Samosa & Tea"]
BASE_FEATURES = ["price_inr", "is_promo", "temperature_c", "rainfall_mm",
                 "students_on_campus", "is_holiday", "is_exam_period", "is_event_day"]
CALENDAR_FEATURES = ["day_of_week", "month", "week_of_year", "is_weekend"]
LAG_FEATURES = ["lag_1", "lag_7", "rolling_mean_7"]


def load_data(path):
    """Read the CSV, parse dates and sort chronologically."""
    df = pd.read_csv(path, parse_dates=["date"])
    return df.sort_values(["date", "item"]).reset_index(drop=True)


def clean_data(df):
    """Remove duplicates, repair weather sensor gaps and validate the target."""
    df = df.drop_duplicates(["date", "item"]).copy()
    weather = df.groupby("date")[["temperature_c", "rainfall_mm"]].first()
    weather["temperature_c"] = weather["temperature_c"].interpolate(
        method="linear", limit_direction="both")           # smooth variable
    weather["rainfall_mm"] = weather.groupby(weather.index.month)[
        "rainfall_mm"].transform(lambda s: s.fillna(s.median()))  # bursty variable
    df = df.drop(columns=["temperature_c", "rainfall_mm"]).merge(
        weather, left_on="date", right_index=True, how="left")
    df[TARGET] = df[TARGET].clip(lower=0)
    return df.sort_values(["date", "item"]).reset_index(drop=True)
```

```
def add_calendar_features(df):
    df = df.copy()
    df["day_of_week"] = df["date"].dt.dayofweek
    df["month"] = df["date"].dt.month
    df["week_of_year"] = df["date"].dt.isocalendar().week.astype(int)
    df["is_weekend"] = (df["day_of_week"] >= 5).astype(int)
    return df


def add_lag_features(df):
    """Past-demand features per menu item (shifted, so no future leakage)."""
    df = df.sort_values(["item", "date"]).copy()
    grp = df.groupby("item")[TARGET]
    df["lag_1"] = grp.shift(1)                                    # yesterday
    df["lag_7"] = grp.shift(7)                                    # same weekday last week
    df["rolling_mean_7"] = grp.transform(lambda s: s.shift(1).rolling(7).mean())
    return df.sort_values(["date", "item"]).reset_index(drop=True)


def build_features(raw_df):
    """Return X (features), y (units sold) and the engineered DataFrame."""
    df = add_lag_features(add_calendar_features(clean_data(raw_df)))
    df = df.dropna(subset=LAG_FEATURES).reset_index(drop=True)   # first 7 days/item
    df["item"] = pd.Categorical(df["item"], categories=ITEMS)
    dummies = pd.get_dummies(df["item"], prefix="item", dtype=int)
    numeric = df[BASE_FEATURES + CALENDAR_FEATURES + LAG_FEATURES]
    X = pd.concat([numeric, dummies], axis=1)
    return X, df[TARGET], df


@dataclass
class Split:
    X_train: pd.DataFrame
    X_test: pd.DataFrame
    y_train: pd.Series
    y_test: pd.Series
    df_train: pd.DataFrame
    df_test: pd.DataFrame
    cutoff: pd.Timestamp


def chronological_split(X, y, df, test_fraction=0.2):
    """Train on the earlier dates, test on the most recent ones (no shuffling)."""
    dates = np.sort(df["date"].unique())
    cutoff = pd.Timestamp(dates[int(len(dates) * (1 - test_fraction))])
    is_train = (df["date"] < cutoff).to_numpy()
    return Split(X[is_train], X[~is_train], y[is_train], y[~is_train],
                 df[is_train], df[~is_train], cutoff)
```

3. Model Tuning and Training (src/train.py)

```
"""Tune, train and save the Random Forest regression model."""
import json
from pathlib import Path
import joblib
import pandas as pd
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, r2_score
from sklearn.model_selection import GridSearchCV, TimeSeriesSplit

RANDOM_STATE = 42
PARAM_GRID = {
    "n_estimators": [100, 200],
    "max_depth": [10, None],
    "min_samples_leaf": [1, 3, 5],
    "max_features": [1.0, 0.5],
}


def tune_model(X_train, y_train, n_splits=5):
    """Grid search with TimeSeriesSplit (each fold validates on a later period)."""
    search = GridSearchCV(
```

```
        RandomForestRegressor(random_state=RANDOM_STATE, n_jobs=-1),
        PARAM_GRID, cv=TimeSeriesSplit(n_splits=n_splits),
        scoring="neg_mean_absolute_error", n_jobs=1)
    search.fit(X_train, y_train)
    return search


def cv_results_table(search, top=5):
    res = pd.DataFrame(search.cv_results_)
    res["cv_mae"] = -res["mean_test_score"]
    res["cv_mae_std"] = res["std_test_score"]
    cols = ["param_n_estimators", "param_max_depth", "param_min_samples_leaf",
            "param_max_features", "cv_mae", "cv_mae_std"]
    out = res.sort_values("cv_mae")[cols].head(top).reset_index(drop=True)
    return out.rename(columns=lambda c: c.replace("param_", ""))


def fit_final_model(X_train, y_train, best_params):
    model = RandomForestRegressor(**best_params, random_state=RANDOM_STATE, n_jobs=-1)
    return model.fit(X_train, y_train)


def training_curve(X_train, y_train, best_params, max_trees=300, step=20,
                   val_fraction=0.2):
    """Diagnostic: train/validation MAE as trees are added.
    Validation = the last 20% of the *training* period (test data is not touched)."""
    n_val = int(len(X_train) * val_fraction)
    Xt, Xv = X_train.iloc[:-n_val], X_train.iloc[-n_val:]
    yt, yv = y_train.iloc[:-n_val], y_train.iloc[-n_val:]
    params = {k: v for k, v in best_params.items() if k != "n_estimators"}
    rf = RandomForestRegressor(n_estimators=step, warm_start=True,
                               random_state=RANDOM_STATE, n_jobs=-1, **params)
    log = []
    for n_trees in range(step, max_trees + 1, step):
        rf.set_params(n_estimators=n_trees).fit(Xt, yt)
        p_tr, p_val = rf.predict(Xt), rf.predict(Xv)
        log.append({"n_trees": n_trees,
                    "train_mae": mean_absolute_error(yt, p_tr),
                    "val_mae": mean_absolute_error(yv, p_val),
                    "val_r2": r2_score(yv, p_val)})
    return pd.DataFrame(log)


def save_model(model, path, best_params, cutoff):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    joblib.dump(model, path)
    meta = {"algorithm": "RandomForestRegressor", "best_params": best_params,
            "train_test_cutoff_date": str(cutoff.date()),
            "features": list(model.feature_names_in_)}
    path.with_suffix(".json").write_text(json.dumps(meta, indent=2, default=str))
```

4. Model Evaluation (src/evaluate.py)

```
"""Evaluation metrics, baseline comparison and kitchen-planning impact."""
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler


def regression_metrics(y_true, y_pred):
    """MAE / RMSE in units, R2, and WAPE (= sum|error| / sum(actual), in %)."""
    y_true, y_pred = np.asarray(y_true, float), np.asarray(y_pred, float)
    return {"MAE": mean_absolute_error(y_true, y_pred),
            "RMSE": float(np.sqrt(mean_squared_error(y_true, y_pred))),
            "R2": r2_score(y_true, y_pred),
            "WAPE_%": 100 * np.abs(y_true - y_pred).sum() / y_true.sum()}


def baseline_predictions(split):
```

```
    """Two reference models the Random Forest must beat."""
    naive = split.X_test["lag_7"].to_numpy()                    # same weekday last week
    linear = make_pipeline(StandardScaler(), LinearRegression())
    linear.fit(split.X_train, split.y_train)
    return {"Naive (last week)": naive,
            "Linear Regression": linear.predict(split.X_test)}


def compare_models(split, rf_pred):
    preds = {"Random Forest": rf_pred, **baseline_predictions(split)}
    table = pd.DataFrame({name: regression_metrics(split.y_test, p)
                          for name, p in preds.items()}).T
    return table.round(3), preds


def planning_impact(y_true, preds):
    """If the kitchen prepares exactly the predicted quantity:
    surplus = prepared but unsold (waste), shortage = demand left unmet."""
    y_true = np.asarray(y_true, float)
    rows = {}
    for name, p in preds.items():
        p = np.round(np.clip(p, 0, None))
        rows[name] = {"surplus_units": np.clip(p - y_true, 0, None).sum(),
                      "shortage_units": np.clip(y_true - p, 0, None).sum()}
    out = pd.DataFrame(rows).T
    out["surplus_%_of_demand"] = 100 * out["surplus_units"] / y_true.sum()
    out["shortage_%_of_demand"] = 100 * out["shortage_units"] / y_true.sum()
    return out.round(2)
```

5. Prediction and Saving Prediction History (src/predict.py)

```
"""Load the trained model, predict a day's demand, and log predictions to SQLite."""
import datetime as dt
import sqlite3
from pathlib import Path
import joblib
import pandas as pd


def load_model(path="models/canteen_rf_model.joblib"):
    return joblib.load(path)


def predict_day(model, X, df, date):
    """Predict units for every menu item on `date` (rows of the engineered frame)."""
    mask = (df["date"] == pd.Timestamp(date)).to_numpy()
    out = df.loc[mask, ["date", "item", "meal_period", "units_sold"]].copy()
    out["predicted_units"] = model.predict(X[mask]).round().astype(int)
    out = out.rename(columns={"units_sold": "actual_units"})
    out["abs_error"] = (out["predicted_units"] - out["actual_units"]).abs()
    return out.reset_index(drop=True)


def log_predictions(pred_df, db_path="database/predictions.db"):
    """Persist predictions (forecast date, item, predicted, actual, error, timestamp)."""
    Path(db_path).parent.mkdir(parents=True, exist_ok=True)
    conn = sqlite3.connect(db_path)
    conn.execute("""CREATE TABLE IF NOT EXISTS prediction_log (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    forecast_date TEXT, item TEXT, predicted_units INTEGER,
                    actual_units INTEGER, abs_error INTEGER, created_at TEXT)""")
    now = dt.datetime.now().isoformat(timespec="seconds")
    conn.executemany(
        "INSERT INTO prediction_log (forecast_date, item, predicted_units, actual_units,"
        " abs_error, created_at) VALUES (?, ?, ?, ?, ?, ?)",
        [(str(r.date.date()), r.item, int(r.predicted_units), int(r.actual_units),
          int(r.abs_error), now) for r in pred_df.itertuples()])
    conn.commit()
    conn.close()


def read_history(db_path="database/predictions.db", limit=10):
    conn = sqlite3.connect(db_path)
```

```
    hist = pd.read_sql_query(
        "SELECT * FROM prediction_log ORDER BY id DESC LIMIT ?", conn, params=(limit,))
    conn.close()
    return hist
```

Output (Step-by-Step Screenshots):

All outputs below were produced by executing the notebook's code cells. JupyterLab could not be installed in the environment used to generate this report, so the cells were run with a scripted executor (tools/build_notebook_and_screenshots.py) and each figure is a rendering of the actual code and captured output of that cell – it is not a screenshot of the JupyterLab window. No value has been edited by hand.

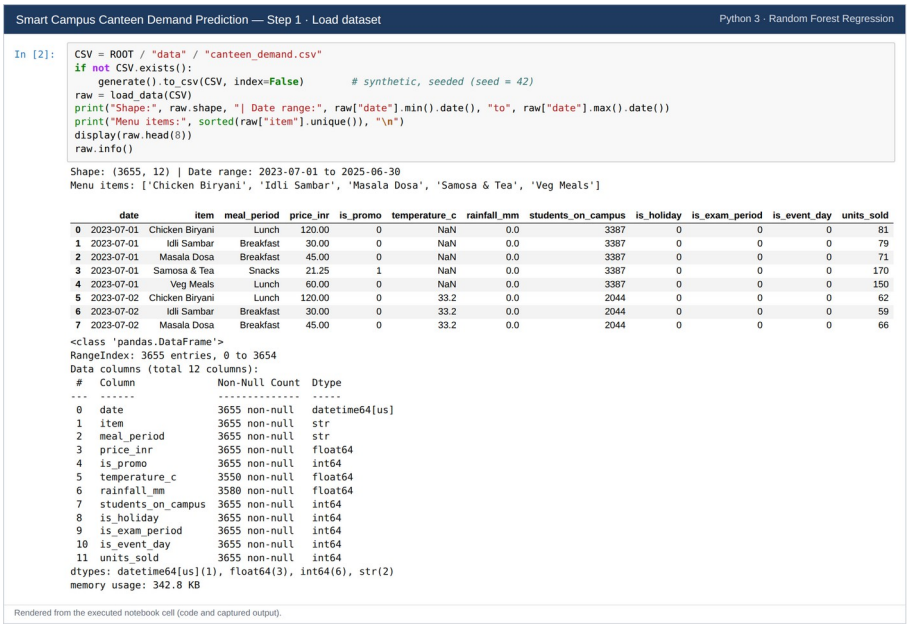


Fig 2: Step 1 — Load dataset: shape, date range, first rows and column information of the canteen sales data (3,655 records, 12 columns)

Smart Campus Canteen Demand Prediction — Step 2 · Preprocessing — Python 3 · Random Forest Regression

In [4]:

```
print("Missing values BEFORE cleaning:")
print(raw.isna().sum()[lambda s: s > 0].to_string())
clean = clean_data(raw)
print("\nMissing values AFTER cleaning:", int(clean.isna().sum().sum()))
X, y, df = build_features(raw)
print(f"Rows after lag features: {len(df)} (first 7 days of each item dropped) | Features: {X.shape[1]}")
display(X.head(3).T)
```

```
Missing values BEFORE cleaning:
temperature_c    105
rainfall_mm       75

Missing values AFTER cleaning: 0
Rows after lag features: 3620 (first 7 days of each item dropped) | Features: 20
```

	0	1	2
price_inr	120.000000	30.000000	45.000000
is_promo	0.000000	0.000000	0.000000
temperature_c	34.200000	34.200000	34.200000
rainfall_mm	0.000000	0.000000	0.000000
students_on_campus	3531.000000	3531.000000	3531.000000
is_holiday	0.000000	0.000000	0.000000
is_exam_period	0.000000	0.000000	0.000000
is_event_day	0.000000	0.000000	0.000000
day_of_week	5.000000	5.000000	5.000000
month	7.000000	7.000000	7.000000
week_of_year	27.000000	27.000000	27.000000
is_weekend	1.000000	1.000000	1.000000
lag_1	122.000000	150.000000	125.000000
lag_7	81.000000	79.000000	71.000000
rolling_mean_7	126.571429	120.714286	107.428571
item_Veg Meals	0.000000	0.000000	0.000000
item_Chicken Biryani	1.000000	0.000000	0.000000
item_Masala Dosa	0.000000	0.000000	1.000000
item_Idli Sambar	0.000000	1.000000	0.000000
item_Samosa & Tea	0.000000	0.000000	0.000000

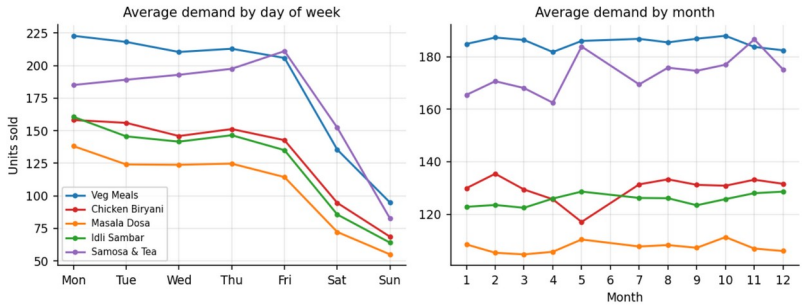
Rendered from the executed notebook cell (code and captured output).

Fig 3: Step 2 — Preprocessing: missing values before and after cleaning, and the 20-feature engineered matrix (first three rows, transposed)

Smart Campus Canteen Demand Prediction — Step 3 · Weekly & seasonal patterns — Python 3 · Random Forest Regression

In [5]:

```
fig = viz.plot_weekday_month(clean); plt.show()
```



Rendered from the executed notebook cell (code and captured output).

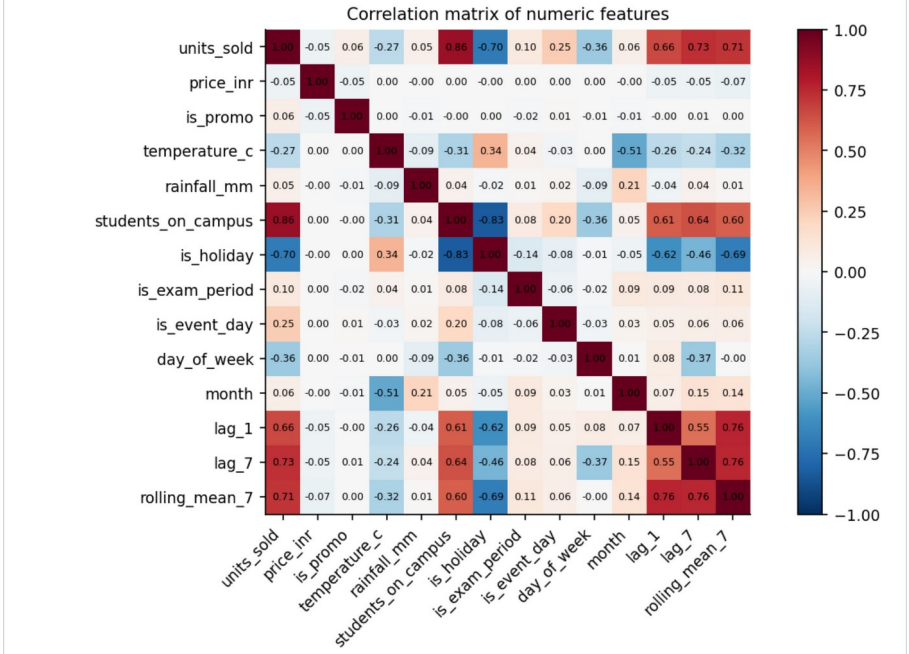
Fig 4: Step 3 — Exploratory analysis: average demand by day of week and by month for each menu item

Smart Campus Canteen Demand Prediction — Step 3 · Feature correlation

Python 3 · Random Forest Regression

In [7]:

```
fig = viz.plot_correlation(df); plt.show()
```



Rendered from the executed notebook cell (code and captured output).

Fig 5: Step 3 — Correlation matrix of the numeric features

Smart Campus Canteen Demand Prediction — Step 4 · Hyper-parameter tuning

Python 3 · Random Forest Regression

In [8]:

```
split = chronological_split(X, y, df, test_fraction=0.2)
print(f"Train: {len(split.X_train)} rows ({split.df_train['date'].min().date()} to {split.df_train['date'].max().date()})")
print(f"Test : {len(split.X_test)} rows ({split.df_test['date'].min().date()} to {split.df_test['date'].max().date()})")
search = tune_model(split.X_train, split.y_train)
print("\nBest parameters:", search.best_params_)
print("Best 5-fold TimeSeriesSplit CV MAE: %.3f units" % -search.best_score_)
display(cv_results_table(search).round(3))
```

```
Train: 2895 rows (2023-07-08 to 2025-02-05)
Test : 725 rows (2025-02-06 to 2025-06-30)

Best parameters: {'max_depth': 10, 'max_features': 1.0, 'min_samples_leaf': 1, 'n_estimators': 200}
Best 5-fold TimeSeriesSplit CV MAE: 13.503 units
```

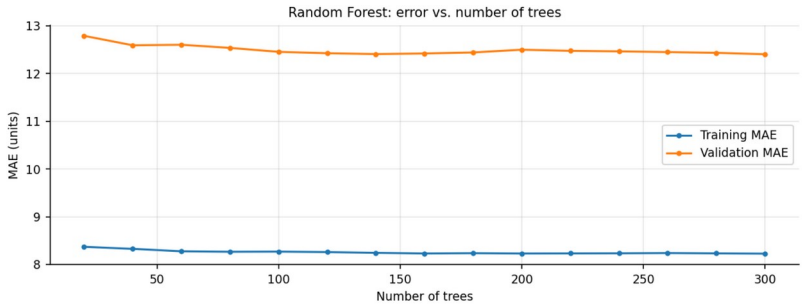
	n_estimators	max_depth	min_samples_leaf	max_features	cv_mae	cv_mae_std
0	200	10	1	1.0	13.503	2.553
1	100	10	1	1.0	13.525	2.565
2	200	None	1	1.0	13.601	2.547
3	200	None	1	0.5	13.614	2.501
4	100	None	1	1.0	13.629	2.553

Rendered from the executed notebook cell (code and captured output).

Fig 6: Step 4 — Chronological train/test split and Random Forest hyper-parameter search (GridSearchCV, 5-fold TimeSeriesSplit)

Smart Campus Canteen Demand Prediction — Step 4 · Training curve — Python 3 · Random Forest Regression

In [9]:
```
log = training_curve(split.X_train, split.y_train, search.best_params_)
fig = viz.plot_training_curve(log); plt.show()
```



Rendered from the executed notebook cell (code and captured output).

Fig 7: Step 4 — Random Forest training and validation error versus number of trees

Smart Campus Canteen Demand Prediction — Step 4 · Training log — Python 3 · Random Forest Regression

In [10]:
```
for r in log.itertuples():
    print(f"[Trees {r.n_trees:3d}/300]  train MAE = {r.train_mae:6.3f} | "
          f"validation MAE = {r.val_mae:6.3f} | validation R2 = {r.val_r2:.4f}")
model = fit_final_model(split.X_train, split.y_train, search.best_params_)
save_model(model, ROOT / "models" / "canteen_rf_model.joblib", search.best_params_, split.cutoff)
print(f"\nFinal model trained on {len(split.X_train)} rows -> models/canteen_rf_model.joblib")
```

```
[Trees  20/300]  train MAE =  8.369 | validation MAE = 12.790 | validation R2 = 0.9262
[Trees  40/300]  train MAE =  8.327 | validation MAE = 12.592 | validation R2 = 0.9290
[Trees  60/300]  train MAE =  8.275 | validation MAE = 12.604 | validation R2 = 0.9284
[Trees  80/300]  train MAE =  8.265 | validation MAE = 12.538 | validation R2 = 0.9292
[Trees 100/300]  train MAE =  8.269 | validation MAE = 12.455 | validation R2 = 0.9300
[Trees 120/300]  train MAE =  8.258 | validation MAE = 12.426 | validation R2 = 0.9307
[Trees 140/300]  train MAE =  8.243 | validation MAE = 12.407 | validation R2 = 0.9302
[Trees 160/300]  train MAE =  8.230 | validation MAE = 12.421 | validation R2 = 0.9303
[Trees 180/300]  train MAE =  8.236 | validation MAE = 12.442 | validation R2 = 0.9299
[Trees 200/300]  train MAE =  8.229 | validation MAE = 12.500 | validation R2 = 0.9293
[Trees 220/300]  train MAE =  8.232 | validation MAE = 12.476 | validation R2 = 0.9295
[Trees 240/300]  train MAE =  8.233 | validation MAE = 12.465 | validation R2 = 0.9295
[Trees 260/300]  train MAE =  8.239 | validation MAE = 12.450 | validation R2 = 0.9295
[Trees 280/300]  train MAE =  8.232 | validation MAE = 12.434 | validation R2 = 0.9297
[Trees 300/300]  train MAE =  8.226 | validation MAE = 12.404 | validation R2 = 0.9301

Final model trained on 2895 rows -> models/canteen_rf_model.joblib
```

Rendered from the executed notebook cell (code and captured output).

Fig 8: Step 4 — Training log (15 checkpoints up to 300 trees) and final model saved to models/canteen_rf_model.joblib

Smart Campus Canteen Demand Prediction — Step 5 · Prediction: regular day — Python 3 · Random Forest Regression

In [11]:
```
test = split.df_test
ctx = ["day_of_week", "temperature_c", "rainfall_mm", "students_on_campus",
       "is_holiday", "is_exam_period", "is_event_day"]

def show_day(date):
    print("Scenario:", pd.Timestamp(date).strftime("%A, %d %b %Y"))
    print(test[test["date"] == date][ctx].iloc[0].to_string(), "\n")
    display(predict_day(model, split.X_test, test, date))

# Sample 1: first dry, regular working Wednesday in the test period
d1 = test[(test.day_of_week == 2) & (test.is_holiday == 0) & (test.is_exam_period == 0) &
          (test.is_event_day == 0) & (test.rainfall_mm == 0)]["date"].iloc[0]
show_day(d1)
```

```
Scenario: Wednesday, 12 Feb 2025
day_of_week              2.0
temperature_c           29.0
rainfall_mm              0.0
students_on_campus    4597.0
is_holiday               0.0
is_exam_period           0.0
is_event_day             0.0
```

	date	item	meal_period	actual_units	predicted_units	abs_error
0	2025-02-12	Chicken Biryani	Lunch	133	147	14
1	2025-02-12	Idli Sambar	Breakfast	145	144	1
2	2025-02-12	Masala Dosa	Breakfast	122	126	4
3	2025-02-12	Samosa & Tea	Snacks	227	185	42
4	2025-02-12	Veg Meals	Lunch	221	207	14

Rendered from the executed notebook cell (code and captured output).

Fig 9: Step 5 — Demand prediction for a regular working day (Wednesday, 12 Feb 2025) with actual sales for comparison

Smart Campus Canteen Demand Prediction — Step 5 · Prediction: campus event day — Python 3 · Random Forest Regression

```
In [12]: # Sample 2: first campus event day in the test period
         d2 = test[test.is_event_day == 1]["date"].iloc[0]
         show_day(d2)
```

```
Scenario: Saturday, 29 Mar 2025
day_of_week               5.0
temperature_c            34.3
rainfall_mm               0.0
students_on_campus     3957.0
is_holiday                0.0
is_exam_period            0.0
is_event_day              1.0
```

	date	item	meal_period	actual_units	predicted_units	abs_error
0	2025-03-29	Chicken Biryani	Lunch	88	114	26
1	2025-03-29	Idli Sambar	Breakfast	108	108	0
2	2025-03-29	Masala Dosa	Breakfast	91	93	2
3	2025-03-29	Samosa & Tea	Snacks	229	198	31
4	2025-03-29	Veg Meals	Lunch	188	193	5

Rendered from the executed notebook cell (code and captured output).

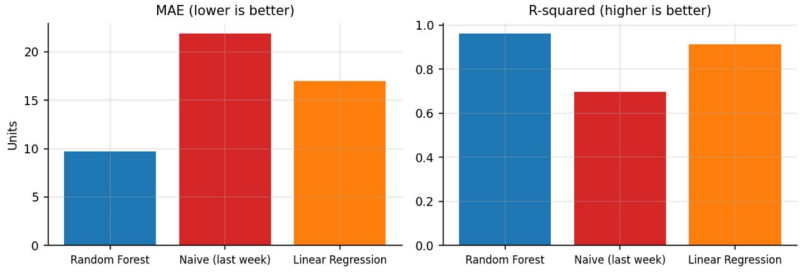
Fig 10: Step 5 — Demand prediction for a campus event day (Saturday, 29 Mar 2025) with actual sales for comparison

Smart Campus Canteen Demand Prediction — Step 6 · Performance metrics & baselines — Python 3 · Random Forest Regression

```
In [13]: pred = model.predict(split.X_test)
         metrics = regression_metrics(split.y_test, pred)
         print("Random Forest - hold-out test metrics")
         print(f"  MAE   : {metrics['MAE']:.3f} units")
         print(f"  RMSE  : {metrics['RMSE']:.3f} units")
         print(f"  R2    : {metrics['R2']:.4f}")
         print(f"  WAPE  : {metrics['WAPE_%']:.2f} %")
         work = split.df_test["is_holiday"].to_numpy() == 0       # robustness check
         m2 = regression_metrics(split.y_test[work], pred[work])
         print(f"Working days only ({work.sum()} rows): MAE {m2['MAE']:.3f} | RMSE {m2['RMSE']:.3f} "
               f"| R2 {m2['R2']:.4f} | WAPE {m2['WAPE_%']:.2f} %\n")
         table, preds = compare_models(split, pred)
         display(table)
         fig = viz.plot_model_comparison(table); plt.show()
```

```
Random Forest - hold-out test metrics
  MAE   : 9.692 units
  RMSE  : 13.851 units
  R2    : 0.9615
  WAPE  : 9.69 %
Working days only (460 rows): MAE 12.785 | RMSE 16.859 | R2 0.8877 | WAPE 8.89 %
```

	MAE	RMSE	R2	WAPE_%
Random Forest	9.692	13.851	0.961	9.691
Naive (last week)	21.866	38.830	0.697	21.863
Linear Regression	16.966	20.888	0.912	16.963

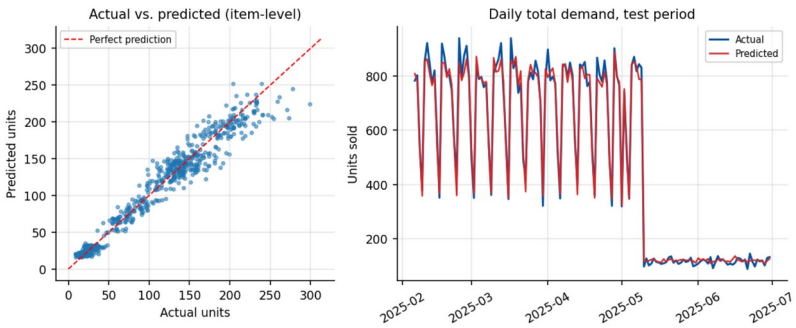


Rendered from the executed notebook cell (code and captured output).

Fig 11: Step 6 — Performance evaluation metrics (MAE 9.69, RMSE 13.85, R^2 0.9615, WAPE 9.69%) and comparison with baseline models

Smart Campus Canteen Demand Prediction — Step 6 · Actual vs. predicted — Python 3 · Random Forest Regression

```
In [14]: fig = viz.plot_actual_vs_pred(split.df_test, pred); plt.show()
```



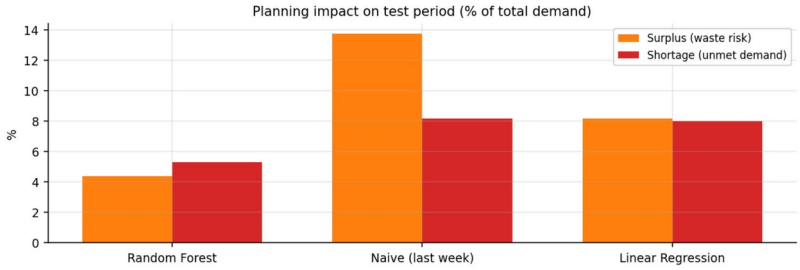
Rendered from the executed notebook cell (code and captured output).

Fig 12: Step 6 — Actual versus predicted demand on the unseen test period

Smart Campus Canteen Demand Prediction — Step 6 · Kitchen planning impact — Python 3 · Random Forest Regression

```
In [16]: impact = planning_impact(split.y_test, preds)
display(impact)
fig = viz.plot_planning_impact(impact); plt.show()
```

	surplus_units	shortage_units	surplus_%_of_demand	shortage_%_of_demand
Random Forest	3187.0	3828.0	4.40	5.28
Naive (last week)	9945.0	5908.0	13.72	8.15
Linear Regression	5915.0	5794.0	8.16	7.99



Rendered from the executed notebook cell (code and captured output).

Fig 13: Step 6 — Kitchen planning impact: surplus and shortage units for each model on the test period

Smart Campus Canteen Demand Prediction — Step 7 · Prediction history (SQLite) — Python 3 · Random Forest Regression

```
In [17]: db = ROOT / "database" / "predictions.db"
db.unlink(missing_ok=True)                    # start with a fresh log
for d in (d1, d2):
    log_predictions(predict_day(model, split.X_test, split.df_test, d), db)
display(read_history(db, limit=10))
```

	id	forecast_date	item	predicted_units	actual_units	abs_error	created_at
0	10	2025-03-29	Veg Meals	193	188	5	2026-10-11T06:37:05
1	9	2025-03-29	Samosa & Tea	198	229	31	2026-10-11T06:37:05
2	8	2025-03-29	Masala Dosa	93	91	2	2026-10-11T06:37:05
3	7	2025-03-29	Idli Sambar	108	108	0	2026-10-11T06:37:05
4	6	2025-03-29	Chicken Biryani	114	88	26	2026-10-11T06:37:05
5	5	2025-02-12	Veg Meals	207	221	14	2026-10-11T06:37:05
6	4	2025-02-12	Samosa & Tea	185	227	42	2026-10-11T06:37:05
7	3	2025-02-12	Masala Dosa	126	122	4	2026-10-11T06:37:05
8	2	2025-02-12	Idli Sambar	144	145	1	2026-10-11T06:37:05
9	1	2025-02-12	Chicken Biryani	147	133	14	2026-10-11T06:37:05

Rendered from the executed notebook cell (code and captured output).

Fig 14: Step 7 — Prediction history log persisted in the SQLite database

Result:

A Smart Campus Canteen Food Demand Prediction system was successfully designed and implemented in Python 3 using pandas, scikit-learn (Random Forest Regression) and an SQLite database, as a JupyterLab notebook with a command-line runner. After cleaning and feature engineering (20 features), the tuned model (200 trees, maximum depth 10) was

trained on the earliest 2,895 records and tested on the latest 725 records (2025-02-06 to 2025-06-30). On this held-out test period the Random Forest achieved an MAE of 9.69 units, RMSE of 13.85 units, R^2 of 0.9615 and WAPE of 9.69%, compared with an MAE of 21.87 and R^2 of 0.697 for the naive "same weekday last week" rule and an MAE of 16.97 and R^2 of 0.912 for Linear Regression. Had the kitchen prepared exactly the predicted quantities, surplus would have been 4.40% and shortage 5.28% of total demand, against 13.72% and 8.15% for the naive rule. Every prediction is logged to a prediction-history database.

These figures must be read with care. First, the dataset is synthetic, so they show that the pipeline works and recovers the simulated patterns, not how accurate it would be in a real canteen. Second, 265 of the 725 test records fall on holiday or vacation days with very low demand, which inflates R^2; on working days only, the same model obtained MAE 12.79, RMSE 16.86, R^2 0.888 and WAPE 8.89%. Third, the students-on-campus feature carries most of the predictive power (importance 0.75, followed by the 7-day rolling mean at 0.14), so in practice it must be known in advance, for example from timetables or ID-card swipes.

In conclusion, a Random Forest regressor given calendar, attendance, weather and recent-sales features forecasts per-item canteen demand far more accurately than a simple weekly rule on the simulated data, and its predictions can be placed in a daily planning routine and logged. The next steps are to retrain on real point-of-sale records, confirm that the attendance inputs are available a day ahead, and extend the model to multi-day forecasts, which requires recursive handling of the lag features.